

Software Architecture

Architecture Means

Bearbeitungshinweise

Ingo Arnold

2025

Inhaltsverzeichnis

1	Architecture Means	3
2	Architecture Principles	3
3	Basic Architecture Concepts	3
4	Architecture Tactics, Styles, and Patterns	5
5	Basic Architectures	6
6	Reference Architectures	7
7	Domain-Driven Design.....	8

1 Architecture Means

Bitte betrachten Sie die (einzige) Folie dieses Kapitels und machen Sie sich deren Aussage klar. Überlegen Sie sich hierbei konkrete Beispiele dafür, wie ausgewählte Architekturmittel (z.B., Architekturprinzipien, Architekturmuster) Einfluss auf die Architektur eines Systems nehmen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6 Architecture Means (WITH WHAT)
- den unter *Bibliography* aufgeführten Literaturhinweisen im Foliensatz.

2 Architecture Principles

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch. Seien Sie in der Lage, zu erklären, was Architekturprinzipien sind und was diese im Softwarearchitektur-Prozess beitragen.

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.1 Architecture Principles
- den unter *Bibliography* aufgeführten Literaturhinweisen im Foliensatz.

3 Basic Architecture Concepts

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What is the primary focus of the object-oriented paradigm?
 - A. Speed of execution
 - B. Bundling data with relevant methods
 - C. Layered systems architecture
 - D. Data-centric design

- Which of the following best describes component orientation?
 - A. Using small, non-reusable classes
 - B. Grouping functionalities into largely self-contained modules
 - C. Hard-coding connections between components
 - D. Focusing on the user interface

- Which architecture concept involves treating programs as data?
 - A. Object Orientation
 - B. Component Orientation
 - C. Meta Programming
 - D. Model-Driven Development

- Model-Driven Development (MDD) relies on:
 - A. Hard-coding system functionality
 - B. Using models as central artifacts for generating system code
 - C. Developing without any models or diagrams
 - D. Focusing specifically on user interfaces

- Which concept helps in achieving platform independence?
 - A. Abstraction
 - B. Object Orientation
 - C. Procedural Approaches
 - D. Layering

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- [*Software Architecture: A Comprehensive Framework and Guide for Practitioners*](#) [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.2 Basic Architecture Concepts
- den unter [*Bibliography*](#) aufgeführten Literaturhinweisen im Foliensatz.

4 Architecture Tactics, Styles, and Patterns

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- Which tactic is focused on improving the system's modifiability?
 - A. Load Balancing
 - B. Error Handling
 - C. Separation of Concerns
 - D. Redundancy
- Which of the following is a structural design pattern?
 - A. Observer
 - B. Factory Method
 - C. Composite
 - D. Strategy
- Which architectural pattern is suitable for systems where components need to be decoupled but still communicate via events?
 - A. Repository
 - B. Client-Server
 - C. Publish-Subscribe
 - D. Layered System

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.3 Architecture Tactics, Styles, and Patterns
- den unter *Bibliography* aufgeführten Literaturhinweisen im Foliensatz.

5 Basic Architectures

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- In a layered architecture, communication between layers should occur:
 - A. Directly between all layers
 - B. Only through defined interfaces
 - C. Using global variables
 - D. Without any restriction
- Middleware is primarily used to:
 - A. Store user data
 - B. Manage communication between distributed components
 - C. Implement business logic
 - D. Design user interfaces
- Which architecture style is ideal for implementing a database-backed web application with a presentation layer?
 - A. Layered Architecture
 - B. Microservices Architecture
 - C. Peer-to-Peer Architecture
 - D. Event-Driven Architecture

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.4 Basic Architectures
- den unter *Bibliography* aufgeführten Literaturhinweisen im Foliensatz.

6 Reference Architectures

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What is a reference architecture?
 - A. A concrete implementation of a software system
 - B. A general blueprint used for solving a specific problem domain
 - C. A collection of software modules
 - D. An algorithm
- Reference architectures are particularly useful because they:
 - A. Replace existing software
 - B. Provide a one-size-fits-all solution
 - C. Offer guidance for structuring (sociotechnical) systems in specific domains
 - D. Are only relevant for small systems
- Which of these could be an example of a reference architecture?
 - A. Layered Architecture
 - B. Microservices
 - C. Business Automation Reference Model
 - D. Factory Method Pattern
- Reference architectures often include:
 - A. Specific programming languages
 - B. Detailed code implementations
 - C. Core system building blocks and their interactions
 - D. Only user interface guidelines

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.

Weitere nützliche Informationen und Erläuterungen finden Sie in:

- *Software Architecture: A Comprehensive Framework and Guide for Practitioners* [Vogel, Arnold et al 2011] (aka: SWA-Buch) – Kapitel:
 - 6.4 Basic Architectures
- den unter *Bibliography* aufgeführten Literaturhinweisen im Foliensatz.

7 Domain-Driven Design

Bitte lesen Sie alle Folien zu diesem Abschnitt aufmerksam durch und beantworten Sie die folgenden Multiple-Choice Fragen:

- What is the main goal of domain-driven design (DDD)?
 - A. Developing software based exclusively on UML diagrams
 - B. Closely coupling business logic and database queries
 - C. Aligning software design with the domain model and the language of the subject domain
 - D. Reusing as many existing frameworks as possible
- What is a domain model in the context of DDD?
 - A. A class diagram that represents the database structure
 - B. A collection of technical frameworks that can be reused
 - C. A conceptual model that describes terms and relationships in the domain
 - D. A list of requirements specified by the customer
- What does the concept of *entity* (reference object) stand for?
 - A. Objects without their own identity, which are described only by their values
 - B. Objects with a stable identity that does not change throughout their entire life cycle
 - C. A collection of objects that are grouped together by rules
 - D. A technical interface for external services
- What does the concept of a *value object* stand for?
 - A. A container for multiple entities
 - B. An object without its own identity, characterized only by its attribute values
 - C. A database table for storing values
 - D. A factory class for creating entities
- What does the concept of *aggregate* stand for?
 - A. A loose collection of arbitrary classes
 - B. A group of entities and value objects that together form a consistency boundary
 - C. A design pattern for generating new objects
 - D. A repository for storing objects

- What does the concept of *service object* stand for?
 - A. A technical interface that abstracts database access
 - B. A special type of entity with extended functionality
 - C. An object that encapsulates a domain-specific operation that does not naturally fit into an entity or value object
 - D. A module that describes external interfaces
- What does the concept of *Factory* stand for?
 - A. An object that serves as the main repository
 - B. A design pattern for consistently creating complex objects or aggregates
 - C. A collection of value objects within an entity
 - D. A component that consumes external services
- What does the concept of *repository* stand for?
 - A. A database instance for persistent storage
 - B. An interface that enables access to aggregates such as a collection of objects
 - C. A factory for creating value objects
 - D. A logging mechanism for domain-specific events

Nutzen Sie ggf. ChatGPT oder andere Quellen, um zentrale Konzepte weiter zu vertiefen.